

AdventureWorks
SQL Çalışma Soruları ve Cevapları
Yönetim Bilişim Sistemleri Bölümü

Microsoft AdventureWorks2022 (OLTP) Örnek Veri Tabanı Üzerine

Konular:

SELECT & WHERE • Aggregation • String Fonksiyonları
JOIN • GROUP BY & HAVING • View Oluşturma
İç İçe Sorgular (Subqueries) • CASE WHEN

1. SELECT ve WHERE

Bu bölüm temel *SELECT* sorgularını, sütun seçimini, koşullu filtrelemeyi (*WHERE*), karşılaştırma operatörlerini ve örüntü eşleşmesini (*LIKE*) içerir.

Soru 1: Production.Product tablosundan tüm ürünlerin Name, ProductNumber ve ListPrice sütunlarını listeleyin.

Çözüm:

```
SELECT Name, ProductNumber, ListPrice
FROM Production.Product;
```

Soru 2: Production.Product tablosundan rengi 'Red' olan ürünleri getirin.

Çözüm:

```
SELECT *
FROM Production.Product
WHERE Color = 'Red';
```

Açıklama: Tek tırnak işareti string değerler için kullanılır.

Soru 3: Production.Product tablosundan ListPrice değeri 100 ile 500 arasında (sınırlar dahil) olan ürünleri listeleyin.

Çözüm:

```
SELECT Name, ListPrice
FROM Production.Product
WHERE ListPrice BETWEEN 100 AND 500;

veya:

SELECT Name, ListPrice
FROM Production.Product
WHERE ListPrice>=100 OR ListPrice<=500;
```

Açıklama: *BETWEEN* operatörü her iki sınırı da dahil eder. ≥ 100 AND ≤ 500 ile aynı sonucu verir.

Soru 4: Person.Person tablosundan FirstName değeri 'A' harfi ile başlayan kişileri *LIKE* kullanarak getirin; sonucu LastName'e göre alfabetik sıralayın.

Çözüm:

```
SELECT FirstName, LastName
FROM Person.Person
WHERE FirstName LIKE 'A%'
ORDER BY LastName ASC;
```

Soru 5: HumanResources.Employee tablosundan JobTitle alanı 'Manager' kelimesini içeren çalışanları listeleyin.

Çözüm:

```
SELECT BusinessEntityID, JobTitle
FROM HumanResources.Employee
WHERE JobTitle LIKE '%Manager%';
```

Veya

```
SELECT BusinessEntityID, JobTitle
FROM HumanResources.Employee
WHERE JobTitle='Manager';
```

Açıklama: İki taraflı % kullanımı, kelimenin metnin neresinde geçtiğini önemsemez.

Soru 6: Sales.SalesOrderHeader tablosundan 01.01.2014 ile 31.12.2014 arasında verilmiş ve TotalDue değeri 10.000'den büyük olan siparişleri OrderDate'e göre azalan sırada listeleyin.

Çözüm:

```
SELECT SalesOrderID, OrderDate, TotalDue
FROM Sales.SalesOrderHeader
WHERE OrderDate BETWEEN '2014-01-01' AND '2014-12-31'
AND TotalDue > 10000
ORDER BY OrderDate DESC;
```

Açıklama: Tarih sabitleri 'YYYY-MM-DD' formatında yazılır. AND birden fazla koşulu birleştirir; DESC azalan sıralama anlamına gelir.

2. Aggregation (Toplama) Fonksiyonları

COUNT, SUM, AVG, MIN, MAX gibi fonksiyonlar birden fazla satırı tek bir özet değere indirger.

Soru 1: Production.Product tablosundaki toplam ürün sayısını bulun.

Çözüm:

```
SELECT COUNT(*) AS ToplamUrun
FROM Production.Product;
```

Açıklama: COUNT(*) tüm satırları sayar (NULL dahil). COUNT(SütunAdı) ise NULL olmayan değerleri sayar.

Soru 2: Production.Product tablosundaki tüm ürünlerin ortalama ListPrice değerini hesaplayın.

Çözüm:

```
SELECT AVG(ListPrice) AS OrtalamaFiyat
FROM Production.Product;
```

Açıklama: AVG fonksiyonu NULL değerleri otomatik olarak hesaplama dışında bırakır.

Soru 3: Sales.SalesOrderHeader tablosundan en yüksek ve en düşük TotalDue değerlerini tek sorguda gösterin.

Çözüm:

```
SELECT MAX(TotalDue) AS EnYuksek,
       MIN(TotalDue) AS EnDusuk
FROM Sales.SalesOrderHeader;
```

Açıklama: MAX ve MIN fonksiyonları aynı sorguda kullanılabilir.

Soru 4: HumanResources.Employee tablosundaki tüm çalışanların toplam VacationHours değerini hesaplayın.

Çözüm:

```
SELECT SUM(VacationHours) AS ToplamIzinSaati
FROM HumanResources.Employee;
```

Soru 5: Sales.SalesOrderDetail tablosunda toplam satılan miktar (SUM(OrderQty)) ve toplam ciro (SUM(LineTotal)) bilgilerini tek sorguda getirin.

Çözüm:

```
SELECT SUM(OrderQty) AS ToplamMiktar,
       SUM(LineTotal) AS ToplamCiro
FROM Sales.SalesOrderDetail;
```

Soru 6: Production.Product tablosunda Color değeri NULL olmayan kaç ürün vardır?

Çözüm:

```
SELECT COUNT(*) AS RenkliUrunSayisi  
FROM Production.Product  
WHERE Color IS NOT NULL;
```

Açıklama: NULL kontrolü için = veya <> değil; IS NULL / IS NOT NULL operatörleri kullanılır.

Alternatif: SELECT COUNT(Color) FROM Production.Product

3. String (Metin) Fonksiyonları

Metin verilerini birleştirme, dönüştürme, kesme ve arama fonksiyonları.

Soru 1: Person.Person tablosundan FirstName ve LastName'i bir boşluk ile birleştirip TamAd takma adıyla getirin.

Çözüm:

```
SELECT FirstName,
       LastName,
       CONCAT(FirstName, ' ', LastName) AS TamAd
FROM Person.Person;
```

Açıklama: CONCAT NULL değerleri boş string olarak işler. Alternatif: FirstName + ' ' + LastName (NULL varsa sonuç NULL olur).

Soru 2: Production.Product tablosundan ürün adlarını büyük harfe çevirip gösterin.

Çözüm:

```
SELECT UPPER(Name) AS UrunAdi
FROM Production.Product;
```

Açıklama: Küçük harf için LOWER() fonksiyonu kullanılır.

Soru 3: Person.Person tablosundan FirstName uzunluğu 8 karakterden fazla olan kişileri listeleyin.

Çözüm:

```
SELECT FirstName,
       LEN(FirstName) AS Uzunluk
FROM Person.Person
WHERE LEN(FirstName) > 8;
```

Açıklama: LEN sondaki boşlukları saymaz. Saymak için DATALENGTH() kullanılır.

Soru 4: Production.Product tablosundan ürün adlarının ilk 10 karakterini ve son 5 karakterini ayrı sütunlar olarak gösterin.

Çözüm:

```
SELECT Name,
       LEFT(Name, 10) AS Ilk10Karakter,
       RIGHT(Name, 5) AS Son5Karakter
FROM Production.Product;
```

Açıklama: İki fonksiyon arasında bir bölüm almak için SUBSTRING(metin, başlangıç, uzunluk) kullanılır.

Soru 5: Production.Product tablosunda adında 'Bike' kelimesi geçen ürünlerde, bu kelimenin kaçınıcı karakterden başladığını gösterin.

Çözüm:

```
SELECT Name,  
        CHARINDEX('Bike', Name) AS BaslangicPozisyonu  
FROM Production.Product  
WHERE Name LIKE '%Bike%';
```

Açıklama: CHARINDEX bulamazsa 0 döner. WHERE filtresi ile sadece bulunan kayıtları görüyoruz.

Soru 6: Person.Person tablosundan FirstName içindeki tüm 'a' harflerini '@' ile değiştirip listeleyin.

Çözüm:

```
SELECT FirstName,  
        REPLACE(FirstName, 'a', '@') AS DegistirilmisAd  
FROM Person.Person;
```

Açıklama: REPLACE fonksiyonu büyük/küçük harfe duyarlıdır (collation ayarına bağlı olarak); hem 'a' hem 'A' değişebilir.

4. JOIN İşlemleri

Birden fazla tabloyu ortak alanlar üzerinden birleştirme. INNER JOIN sadece eşleşenleri, LEFT JOIN sol tablonun tüm satırlarını, , RIGHT JOIN sağ tablonun tüm satırlarını getirir.

Soru 1: Production.Product ve Production.ProductSubcategory tablolarını birleştirip her ürünün adı ile alt kategori adını getirin.

Çözüm:

```
SELECT p.Name AS UrunAdi,  
       ps.Name AS AltKategori  
FROM Production.Product p  
INNER JOIN Production.ProductSubcategory ps  
       ON p.ProductSubcategoryID = ps.ProductSubcategoryID;
```

Açıklama: Tablo takma adları (alias: p, ps) sorguyu kısaltır. INNER JOIN sadece her iki tabloda da eşleşen kayıtları getirir.

Soru 2: Product, ProductSubcategory ve ProductCategory tablolarını birleştirip ürün adı, alt kategori adı, ana kategori adı ve ListPrice bilgilerini gösterin.

Çözüm:

```
SELECT p.Name AS UrunAdi,  
       ps.Name AS AltKategori,  
       pc.Name AS AnaKategori,  
       p.ListPrice  
FROM Production.Product p  
INNER JOIN Production.ProductSubcategory ps  
       ON p.ProductSubcategoryID = ps.ProductSubcategoryID  
INNER JOIN Production.ProductCategory pc  
       ON ps.ProductCategoryID = pc.ProductCategoryID;
```

Açıklama: Birden fazla JOIN'i zincirleme kullanabilirsiniz. Her JOIN'in kendi ON koşulu olur.

Soru 3: Çalışanların ad, soyad, ünvan ve şu anki departman adını listeleyin (EndDate IS NULL → aktif görev).

Çözüm:

```
SELECT pe.FirstName,  
       pe.LastName,  
       e.JobTitle,  
       d.Name AS Departman  
FROM HumanResources.Employee e  
INNER JOIN Person.Person pe  
       ON e.BusinessEntityID = pe.BusinessEntityID  
INNER JOIN HumanResources.EmployeeDepartmentHistory edh  
       ON e.BusinessEntityID = edh.BusinessEntityID  
INNER JOIN HumanResources.Department d
```



```
ON edh.DepartmentID = d.DepartmentID  
WHERE edh.EndDate IS NULL;
```

Açıklama: EndDate IS NULL koşulu, çalışanın halen o departmanda çalıştığını gösterir.

Soru 4: LEFT JOIN ile tüm ürünleri ve varsa toplam sipariş miktarını gösterin (hiç sipariş edilmemiş ürünler de listede olsun).

Çözüm:

```
SELECT p.Name,  
       SUM(sod.OrderQty) AS ToplamSatılanMiktar  
FROM Production.Product p  
LEFT JOIN Sales.SalesOrderDetail sod  
       ON p.ProductID = sod.ProductID  
GROUP BY p.Name  
ORDER BY ToplamSatılanMiktar DESC;
```

Açıklama: LEFT JOIN sol tablodaki tüm satırları getirir; sağ tarafta eşleşme yoksa NULL gelir. Hiç satılmamış ürünler için SUM = NULL olur.

5. GROUP BY ve HAVING

GROUP BY satırları belirli kolon(lar) ile gruplayıp her grup için aggregation hesaplar. *HAVING* ise grup düzeyinde filtreleme yapar (*WHERE*'den farkı budur).

Soru 1: Her renk için kaç ürün olduğunu listeleyin (NULL renkler hariç).

Çözüm:

```
SELECT Color,  
       COUNT(*) AS UrunSayisi  
FROM Production.Product  
WHERE Color IS NOT NULL  
GROUP BY Color;
```

Açıklama: *WHERE* *GROUP BY*'dan ÖNCE çalışır; satırları gruplanmadan önce filtreler.

Soru 2: Her ürün için toplam satış miktarını gösterin; sonucu büyükten küçüğe sıralayın.

Çözüm:

```
SELECT ProductID,  
       SUM(OrderQty) AS ToplamMiktar  
FROM Sales.SalesOrderDetail  
GROUP BY ProductID  
ORDER BY ToplamMiktar DESC;
```

Soru 3: Her müşteri için sipariş sayısını gösterin ve 5'ten fazla sipariş veren müşterileri filtreleyin.

Çözüm:

```
SELECT CustomerID,  
       COUNT(*) AS SiparisSayisi  
FROM Sales.SalesOrderHeader  
GROUP BY CustomerID  
HAVING COUNT(*) > 5;
```

Açıklama: *HAVING* grup oluştuktan SONRA çalışır; aggregation sonuçlarına göre filtreleme yapar.

Soru 4: Her alt kategori için ortalama ListPrice değerini hesaplayın; ortalaması 1000'den büyük olanları listeleyin.

Çözüm:

```
SELECT ProductSubcategoryID,  
       AVG(ListPrice) AS OrtalamaFiyat  
FROM Production.Product  
WHERE ProductSubcategoryID IS NOT NULL
```

```
GROUP BY ProductSubcategoryID  
HAVING AVG(ListPrice) > 1000;
```

Açıklama: WHERE ile NULL alt kategorileri eliyoruz, HAVING ile ortalama 1000'den büyük olanları seçiyoruz. İkisinin sırası önemli.

Soru 5: Her JobTitle için çalışan sayısını gösterin; 2 veya daha fazla çalışanı olan ünvanları getirin.

Çözüm:

```
SELECT JobTitle,  
       COUNT(*) AS CalisanSayisi  
FROM HumanResources.Employee  
GROUP BY JobTitle  
HAVING COUNT(*) >= 2;
```

Soru 6: Yıl bazında toplam ciroyu ve sipariş sayısını gösterin; cirosu 10 milyondan fazla olan yılları getirin.

Çözüm:

```
SELECT YEAR(OrderDate) AS Yil,  
       SUM(TotalDue) AS ToplamCiro,  
       COUNT(*) AS SiparisSayisi  
FROM Sales.SalesOrderHeader  
GROUP BY YEAR(OrderDate)  
HAVING SUM(TotalDue) > 10000000  
ORDER BY Yil;
```

Açıklama: YEAR() fonksiyonu tarih sütunundan yılı çıkarır. GROUP BY içinde fonksiyon kullanılabilir.

6. View (Görüntü Tablosu) Oluşturma

View bir *SELECT* sorgusunun saklanmış halidir; tablo gibi sorgulanabilir, ama gerçek veriyi tutmaz. Karmaşık sorguları sadeleştirir, güvenlik katmanı sağlar.

Soru 1: vw_PahalıUrunler: ListPrice > 1000 olan ürünlerin Name, ProductNumber, Color, ListPrice bilgilerini listeleyen view.

Çözüm:

```
CREATE VIEW vw_PahalıUrunler AS
SELECT Name,
       ProductNumber,
       Color,
       ListPrice
FROM Production.Product
WHERE ListPrice > 1000;
```

Açıklama: Oluşturulduktan sonra şöyle kullanılır: *SELECT * FROM vw_PahalıUrunler;*

Soru 2: vw_AktifÇalışanListesi: CurrentFlag = 1 olan çalışanların ad, soyad, JobTitle, HireDate bilgilerini gösteren view.

Çözüm:

```
CREATE VIEW vw_AktifÇalışanListesi AS
SELECT pe.FirstName,
       pe.LastName,
       e.JobTitle,
       e.HireDate
FROM HumanResources.Employee e
INNER JOIN Person.Person pe
      ON e.BusinessEntityID = pe.BusinessEntityID
WHERE e.CurrentFlag = 1;
```

Soru 3: vw_UrünKategoriDetay: Ürün + alt kategori + ana kategori bilgilerini birleştiren view.

Çözüm:

```
CREATE VIEW vw_UrünKategoriDetay AS
SELECT p.Name AS ÜrünAdı,
       ps.Name AS AltKategori,
       pc.Name AS AnaKategori,
       p.ListPrice
FROM Production.Product p
INNER JOIN Production.ProductSubcategory ps
      ON p.ProductSubcategoryID = ps.ProductSubcategoryID
INNER JOIN Production.ProductCategory pc
      ON ps.ProductCategoryID = pc.ProductCategoryID;
```

Açıklama: Bu view sayesinde 3 tabloyu birleştiren JOIN'i her seferinde yazmak zorunda kalmazsınız.

Soru 4: vw_MusteriSiparisOzet: Her müşteri için toplam sipariş sayısı ve toplam ciroyu gösteren view.

Çözüm:

```
CREATE VIEW vw_MusteriSiparisOzet AS
SELECT CustomerID,
       COUNT(*) AS SiparisSayisi,
       SUM(TotalDue) AS ToplamCiro
FROM Sales.SalesOrderHeader
GROUP BY CustomerID;
```

Soru 5: vw_SatisTemsilcisiPerformans: Her satış temsilcisinin ad-soyad, bölge, yıllık satış (SalesYTD) ve geçen yıl satışını gösteren view.

Çözüm:

```
CREATE VIEW vw_SatisTemsilcisiPerformans AS
SELECT pe.FirstName,
       pe.LastName,
       sp.TerritoryID,
       sp.SalesYTD,
       sp.SalesLastYear
FROM Sales.SalesPerson sp
INNER JOIN Person.Person pe
    ON sp.BusinessEntityID = pe.BusinessEntityID;
```

Soru 6: vw_HicSatilmamisUrunler: Hiç sipariş edilmemiş ürünleri listeleyen view (LEFT JOIN + NULL kontrolü).

Çözüm:

```
CREATE VIEW vw_HicSatilmamisUrunler AS
SELECT p.ProductID,
       p.Name,
       p.ProductNumber
FROM Production.Product p
LEFT JOIN Sales.SalesOrderDetail sod
    ON p.ProductID = sod.ProductID
WHERE sod.ProductID IS NULL;
```

Açıklama: LEFT JOIN sonrası WHERE ... IS NULL klasik 'eşleşmeyenler' kalıbıdır.

7. İç İçe Sorgular (Subqueries)

Bir sorgunun içinde başka bir sorgu çalıştırma. WHERE, FROM ve SELECT bölümlerinde kullanılabilir. Correlated subquery dış sorgudaki her satıra bağlı olarak yeniden çalışır.

Soru 1: Ortalama ListPrice'tan daha pahalı olan ürünleri listeleyin.

Çözüm:

```
SELECT Name,  
       ListPrice  
FROM Production.Product  
WHERE ListPrice > (  
    SELECT AVG(ListPrice)  
    FROM Production.Product  
);
```

Açıklama: Alt sorgu önce çalışır ve tek bir değer üretir; dış sorgu bu değerle karşılaştırma yapar.

Soru 2: En yüksek TotalDue değerine sahip siparişin tüm bilgilerini getirin.

Çözüm:

```
SELECT *  
FROM Sales.SalesOrderHeader  
WHERE TotalDue = (  
    SELECT MAX(TotalDue)  
    FROM Sales.SalesOrderHeader  
);
```

Açıklama: Eğer aynı maksimum değere sahip birden fazla sipariş varsa hepsi gelir.

Soru 3: Hiç sipariş edilmemiş ürünleri NOT IN ile listeleyin.

Çözüm:

```
SELECT ProductID,  
       Name  
FROM Production.Product  
WHERE ProductID NOT IN (  
    SELECT DISTINCT ProductID  
    FROM Sales.SalesOrderDetail  
);
```

Açıklama: DİKKAT: Eğer alt sorgu NULL döndürürse NOT IN beklenmedik sonuç verir. Daha güvenli alternatif: NOT EXISTS veya LEFT JOIN ... IS NULL.

Soru 4: FROM içinde subquery (derived table) ile en yüksek ciroya sahip yılı bulun.

Çözüm:

```
SELECT TOP 1 Yil, ToplamCiro
FROM (
    SELECT YEAR(OrderDate) AS Yil,
           SUM(TotalDue) AS ToplamCiro
    FROM Sales.SalesOrderHeader
    GROUP BY YEAR(OrderDate)
) AS YillikCiro
ORDER BY ToplamCiro DESC;
```

Açıklama: FROM içindeki subquery'ye derived table denir ve mutlaka takma ad (alias) verilmelidir (örn. AS YillikCiro).

8. CASE WHEN İfadeleri

CASE WHEN, SQL'in 'IF-ELSE' yapısıdır. Veriyi kategorize etmek, koşullu sütun üretmek, pivot benzeri toplamalar yapmak için kullanılır. İki sözdizimi vardır: Simple CASE (CASE sütun WHEN değer THEN ...) ve Searched CASE (CASE WHEN koşul THEN ...).

Soru 1: Ürünleri ListPrice değerine göre 5 fiyat segmentine ayırın: Ücretsiz, Ucuz, Orta, Pahalı, Lüks.

Çözüm:

```
SELECT Name,
       ListPrice,
       CASE
           WHEN ListPrice = 0 THEN 'Ücretsiz'
           WHEN ListPrice < 100 THEN 'Ucuz'
           WHEN ListPrice BETWEEN 100 AND 500 THEN 'Orta'
           WHEN ListPrice BETWEEN 500 AND 2000 THEN 'Pahalı'
           ELSE 'Lüks'
       END AS FiyatSegmenti
FROM Production.Product;
```

Açıklama: WHEN koşulları yukarıdan aşağı kontrol edilir; ilk eşleşen sonuç döner. ELSE eşleşme olmazsa devreye girer.

Soru 2: Çalışan cinsiyet kodunu Türkçe yazıya dönüştürün ('M'→Erkek, 'F'→Kadın). Simple CASE kullanın.

Çözüm:

```
SELECT BusinessEntityID,
       JobTitle,
       Gender,
       CASE Gender
           WHEN 'M' THEN 'Erkek'
           WHEN 'F' THEN 'Kadın'
           ELSE 'Belirtilmemiş'
       END AS Cinsiyet
FROM HumanResources.Employee;
```

Açıklama: Simple CASE: 'CASE sütun WHEN değer THEN ...' formatı, sadece eşitlik kontrolü için kullanışlıdır.

Soru 3: Sipariş tutarına göre indirim oranı uygulayın: 100K üstü %15, 50K-100K %10, 10K-50K %5, altı indirimli.

Çözüm:

```
SELECT SalesOrderID,
       TotalDue,
       CASE
```



```

        WHEN TotalDue >= 100000 THEN '%15'
        WHEN TotalDue >= 50000 THEN '%10'
        WHEN TotalDue >= 10000 THEN '%5'
        ELSE '%0'
    END AS IndirimOrani,
    CASE
        WHEN TotalDue >= 100000 THEN TotalDue * 0.85
        WHEN TotalDue >= 50000 THEN TotalDue * 0.90
        WHEN TotalDue >= 10000 THEN TotalDue * 0.95
        ELSE TotalDue
    END AS IndirimliTutar
FROM Sales.SalesOrderHeader;

```

Açıklama: Sıralama önemli: En yüksek koşul en üstte olmalı. Aksi halde her sipariş ilk WHEN'e takılır.

Soru 4: GROUP BY ile birlikte CASE: Yıl bazında ilk yarıyıl ve ikinci yarıyıl sipariş sayılarını ayrı sütunlar olarak gösterin.

Çözüm:

```

SELECT YEAR(OrderDate) AS Yil,
       SUM(CASE
           WHEN MONTH(OrderDate) BETWEEN 1 AND 6 THEN 1
           ELSE 0
       END) AS IlkYari,
       SUM(CASE
           WHEN MONTH(OrderDate) BETWEEN 7 AND 12 THEN 1
           ELSE 0
       END) AS IkinciYari,
       COUNT(*) AS ToplamSiparis
FROM Sales.SalesOrderHeader
GROUP BY YEAR(OrderDate)
ORDER BY Yil;

```

Açıklama: Aggregation içinde CASE çok güçlü bir tekniktir: Şartlı sayım yapmamızı sağlar (conditional aggregation).

Soru 5: PIVOT mantığı: Renge göre toplam ListPrice'i ayrı sütunlar olarak gösterin (Red, Black, Silver, Renksiz).

Çözüm:

```

SELECT
    SUM(CASE WHEN Color = 'Red' THEN ListPrice ELSE 0 END) AS
    KirmiziToplam,
    SUM(CASE WHEN Color = 'Black' THEN ListPrice ELSE 0 END) AS
    SiyahToplam,

```

```
SUM(CASE WHEN Color = 'Silver' THEN ListPrice ELSE 0 END) AS
GumusToplam,
SUM(CASE WHEN Color IS NULL THEN ListPrice ELSE 0 END) AS
RenksizToplam
FROM Production.Product;
```

Açıklama: Bu teknik PIVOT operatörü olmadan da satır verisini sütuna çevirmenize olanak verir; raporlama sorgularında çok yaygındır.

Soru 6: İleri seviye: Her satış temsilcisinin SalesYTD performansını 'Hedef Üstü / Normal / Hedef Altı' olarak kategorize edin (kotaya göre karşılaştırma).

Çözüm:

```
SELECT pe.FirstName,
       pe.LastName,
       sp.SalesQuota,
       sp.SalesYTD,
       CASE
           WHEN sp.SalesQuota IS NULL THEN 'Kota Atanmamış'
           WHEN sp.SalesYTD >= sp.SalesQuota * 1.10 THEN 'Hedef
Üstü (%10+)'
           WHEN sp.SalesYTD >= sp.SalesQuota THEN 'Hedef
Tutturuldu'
           WHEN sp.SalesYTD >= sp.SalesQuota * 0.80 THEN 'Hedefe
Yakın'
           ELSE 'Hedef Altı'
       END AS Performans
FROM Sales.SalesPerson sp
INNER JOIN Person.Person pe
    ON sp.BusinessEntityID = pe.BusinessEntityID
ORDER BY sp.SalesYTD DESC;
```

Açıklama: Gerçek iş senaryolarında CASE genelde KPI takibi, performans skortlama ve müşteri segmentasyonu için kullanılır.